CHRIST (Deemed to be University), Bengaluru
Department of Computer Science

Project Diary

Course: CSC481-5, Computer Science Project
Semester V

ThinkStack

A Record of Decisions, Implementations and Corrections

Submitted by

Sl. No.	Student Name	Register No.
1.	Aditya Mehta	2440204
2.	Jitvan Chadha	2440222
3.	Ritesh K R	2440233

Guide: Dr. New Begin
Class: 5BSC CS
Repository: `github.com/get-thinkstack/ThinkStack`

July 2026

About this Diary

This diary records the development of ThinkStack from 5 June to 5 August 2026. It is reconstructed from the repository itself: 237 commits, 43 merged pull requests, 40 recorded architecture decisions, and the published releases. It is not written from memory.

Two conventions are observed throughout. First, every quantity was measured or counted, never estimated. Second, decisions that turned out to be wrong are recorded alongside those that were right, because a diary that contains only successes teaches nothing.

The work was carried out jointly by the three team members. Entries describe what was decided and built rather than who performed each action.

Team: Aditya Mehta (2440204), Jitvan Chadha (2440222), Ritesh K R (2440233)

Contents

1 Timeline at a Glance

Date	Phase	Event
5 Jun	Sprint I	repository created
17 Jun	Sprint I	project renamed ScholarLens to ThinkStack; decision log started
19 Jun	Sprint I	desktop shell introduced
20 Jun	Sprint I	encryption implemented
23 Jun	Sprint I	LaTeX editor and auto-healing compiler
24 Jun	Sprint I	interface unified; Sprint I closes
7 Jul	Sprint II	download page
10 Jul	Sprint II	cross-platform builds and CI
21 Jul	Sprint II	hardware-aware model routing
22 Jul	Sprint II	in-application updates
25 Jul	Sprint II	test suite introduced; Rust startup diagnosis
27 Jul	Sprint II	first public release, v0.1.0
29 Jul	Sprint II	v0.1.1, then v1.0.0
30 Jul	Sprint II	v1.0.0 found not to start; root cause analysis and correction
30 Jul	Sprint II	TeX engine bundled; validated beta published
31 Jul	Sprint III	release pipeline consolidated; every dependency pinned
1 to 3 Aug	Sprint III	LitGraph: search, analysis and gaps become one canvas
1 Aug	Sprint III	stale interface after update traced to the webview cache
2 Aug	Sprint III	capability model extracted; Bench introduced
4 Aug	Sprint III	auto-update failure diagnosed; installs on a dead channel rescued
4 Aug	Sprint III	beta v1.9.0 published
4 to 5 Aug	Sprint III	model registry, task router, and Hugging Face acquisition
5 Aug	Sprint III	baseline chosen by measurement; render tests added

2 Sprint I: Building the Application

2.1 5 to 16 June: foundations

The repository was created on 5 June. The first two weeks established the project structure, an initial working demonstration, and the first iteration of the interface.

2.2 17 June: naming, audit, and the decision log

Three things happened on the same day that shaped everything afterwards.

The project was renamed from ScholarLens to ThinkStack. A backend audit corrected a set of defects found by reading rather than by testing. Most consequentially, an architecture decision log was started.

> **Decision recorded.** Run entirely on the user's device, with no hosted language model. The reasoning: a researcher working with unpublished results or institutionally restricted material cannot upload it, and a student project has no budget for per-call inference. This single decision determined local inference, quantised models, a frozen runtime, and eventually a bundled typesetting engine.

The log reached 32 entries by the end of the project. Its value was not apparent at the time; it became apparent during the dependency audit in the final week, when the reasoning behind earlier choices could be read rather than reconstructed.

2.3 19 to 21 June: desktop shell and encryption

The application was wrapped in a desktop shell. The shell starts the backend as a supervised child process and stops it when the window closes, so no orphaned process is left holding a port.

Encryption was implemented with Argon2id for key derivation and AES-256-GCM for the ciphertext.

> **Decision recorded.** Use authenticated encryption rather than an unauthenticated mode. Tampering with a stored paper is then detected at decryption instead of producing plausible but corrupted output, which for research material is the difference between an error and a silent falsification.

2.4 23 to 24 June: the paper writer

The LaTeX editor was implemented, with a compiler designed to be forgiving. A small language model produces markup that is frequently almost correct, so the compiler inserts missing package declarations, wraps bare fragments into complete documents, and continues past recoverable errors. When a pass produces no PDF, the failing environment is progressively neutralised and compilation is retried, so one malformed figure does not cost the author the paper.

Sprint I closed on 24 June with every feature working from a source checkout.

3 Sprint II: Making It Usable by Others

3.1 7 to 10 July: distribution

A download page was produced, and cross-platform builds were automated. The build configuration was made data-driven, so that adding an operating system is an edit to a configuration file rather than to build logic.

3.2 20 to 22 July: hardware awareness

Model selection was made dependent on the machine rather than fixed.

> **Decision recorded.** Size the model against *available* memory rather than installed memory. A user has a browser and an editor open. Sizing against total memory produces a process that is terminated by the operating system partway through an analysis, which is worse than producing a slightly weaker answer.

A related decision concerned graphics hardware. The diagnosis reports the GPU but enables offload only for one specific class of device.

> **Decision recorded.** Report the GPU, but treat all but CUDA devices with sufficient memory as unusable. The inference library shipped in the installer is a CPU build, so attempting offload elsewhere fails at model load. Reporting a capability the runtime cannot use converts a working configuration into a crash.

3.3 25 July: testing and the Rust rewrite

An automated test suite was introduced. Startup diagnosis was moved from Python into the desktop shell, written in Rust, because the Python implementation imported a deep learning framework purely to determine whether a GPU existed, which cost several seconds on every launch.

> **Decision recorded, with a limit.** Hardware diagnosis moved to Rust because it was on the startup path and slow. Model discovery was measured at approximately 9 milliseconds and was left in Python, because the same argument does not apply to code that is already fast. The principle recorded was to move work for a measured reason, not because a language seems faster.

3.4 27 to 29 July: first releases

Version 0.1.0 was published on 27 July, the first release to build on all three operating systems. Two defects had blocked it, both worth recording because neither was reproducible locally.

- The Windows build failed at model download. A JSON tool on that platform emits carriage returns, which left an invalid character on the end of a URL. The transfer tool rejected it. Linux and macOS were unaffected, so the defect existed only on the platform none of the team used daily.
- The macOS build failed at code signing. An absent optional credential was passed as an empty string, so the signing tool attempted to import an empty certificate rather than skipping signing.

Version 1.0.0 followed on 29 July, alongside a decision to reduce the installer size.

Decision recorded. Ship only the smaller model and offer the larger one with the user's consent, reusing weights the user may already have through other local tools. This removed approximately one gigabyte from every installer. Matching required a canonical naming key, because the same weights are named differently by each runtime, and comparing filenames caused the application to offer a download the user did not need.

3.5 30 July: the release that did not work

Observed. The published v1.0.0 installer, downloaded and run, displayed a loading screen indefinitely. One tester waited approximately 200 seconds. Continuous integration had been green throughout.

This is the most instructive day of the project.

3.5.1 Diagnosis

The startup path was instrumented before anything was changed, on the reasoning that the failure could not be fixed while it was invisible. The instrumentation identified the cause in a single run.

Three defects compounded:

1. The packaging framework installs the executable and its resources into different directories. The lookup searched only beside the executable, found nothing, and fell back to the development path of launching a system Python interpreter.
2. The Linux application image exports an environment variable that redirects a Python interpreter's library search into the mounted image. A frozen interpreter ignores this; a system interpreter does not, and it terminated before running any project code.
3. The loading screen retried without limit and had no error state, so a backend that died in under a second looked exactly like one that was slow.

3.5.2 What the same investigation uncovered

- The embedding weights had never been placed in the bundle. The first document ingested on an installed copy attempted a network download, in an application whose entire premise is that it needs no network. The code's own documentation stated the opposite.
- The model directory was derived from the current working directory, which the application image changes before launching. This was incorrect on every operating system for every installed copy; it worked only when launched from a developer checkout.

- The window terminated approximately one second after appearing, from heap corruption in a graphics path of the system webview. Every backend check passed while this happened, because the backend was healthy and only the window was gone.

3.5.3 The correction that mattered most

Rather than continue fixing defects individually, every external dependency was enumerated by walking all imports and every subprocess call across 51 backend modules.

> **Result.** Exactly one unbundled runtime dependency existed: the TeX engine. Everything else was already inside the bundle. Bundling that engine closed the entire category rather than one instance of it.

The pattern the audit exposed was that the absent TeX engine, the absent embedding weights and the incorrect model path were the same defect: an assumption that a user's machine resembles a developer's.

3.6 30 July: bundling the typesetting engine

The engine was measured before it was adopted: a 58 MB binary and a package cache of approximately 47 MB, against roughly 1.1 GB of headroom under the hosting platform's asset limit.

The first attempt worked locally and failed on the Linux build server, which produced the most transferable lesson of the project.

> **Cause.** The chosen engine version required a newer system C library than the build platform provides, so it never executed at all, failing in 15 milliseconds. The build step recorded a warning and continued, so the installer was assembled with an engine that had no package cache. The same binary would have reached users of long-term-support distributions, who would have had a paper writer that silently could not compile.

> **Correction.** The engine version was chosen by measuring the minimum system library each release requires, rather than by taking the newest release. The selected version runs on the build platform and on older user systems, and reduced the bundled payload from 104 MB to 70 MB. Equally important, the build step now prints the engine's actual error and stops the build, because suppressing that error is what made the diagnosis expensive.

3.7 30 July: verified delivery

A validated beta was published to all three operating systems, with every automated check passing, including compilation of a PDF by the packaged application.

4 Sprint III: Correctness of What Was Delivered

Sprint II ended with software that installed. Sprint III began with the discovery that installing is not the same as remaining correct, and most of its work was caused by faults that only appear after a second release exists.

4.1 31 July to 1 August: the pipeline, and what a green tick means

Five release workflows had accumulated, differing only in what started them and how the version was derived; their build and publish halves were identical. They were consolidated into one, with the channel decided by which branch received the merge.

Two failures during this period shared a cause. A macOS launch check had been ordered before the macOS build and marked `continue-on-error`, so it reported success for a run in which it never executed. Separately, an unpinned transitive dependency updated overnight and broke all three platforms from a commit that changed no application code. Both were cases of a signal that appeared to mean verification and did not.

A third fault was subtler. Users who updated correctly continued to see the previous interface, because the embedded webview served `index.html` from its own cache, which outlives the application. The correction was to serve the entry document with `no-store` while allowing content-hashed assets to remain immutable.

4.2 1 to 3 August: LitGraph

The gap finder, search, and summarisation had been separate screens over the same corpus and much of the same computation. They were consolidated into one canvas, in which the literature is presented as a connected structure rather than as three lists: a question is asked once, the passages that answer it are shown, the papers containing them are shown in relation to one another, and the places where the literature is thin are marked on the same surface.

Analysis was moved off the request path and precomputed at ingestion, because a canvas that must summarise before it can draw is a canvas that appears broken. Selecting a node opens the paper itself, with the passages the map relied on marked in it, so that a claim on the canvas can be checked against its source without leaving the screen.

This was the largest single feature of the project and closes the consolidation that had been recorded as planned work since Sprint I.

4.3 2 August: separating measurement from decision

Hardware classification existed twice: the Rust shell measured the machine at startup, and the Python backend measured it again. The two disagreed, and the Rust answer silently won, leaving the Python path authoritative-looking and dead.

The responsibilities were separated. Rust measures; a single Python module converts those measurements into decisions — context size, offload, token budgets, and whether a document must be summarised in several passes. One consequence was immediate: graphics offload had been decided by a test for CUDA and dedicated video memory, which Apple Silicon can never satisfy despite having a usable GPU, so every Mac had been running on the processor alone.

The same reorganisation gave the interface a place to show its findings, and *Bench* was introduced beside Library, LitGraph and Scribe: what the machine can run, and what it runs it with.

4.4 4 August: an update mechanism that could not succeed

A tester reported that the update function did nothing on macOS. The investigation found three faults in sequence, each concealing the next.

The update endpoint is compiled into the application and cannot be changed afterwards. A rolling release tag had been renamed some days earlier, so every installation built during that window continued to poll an address that still existed and still advertised the build they already had. Separately, the manifest carried a pre-release version string while the application reported the plain version, and semantic versioning ranks a pre-release below its own release — so even a correctly aimed installation was told it was current.

The stranded installations could not be reached by any change to the source, because the address they poll is fixed in the binary they are running. They were recovered by publishing a manifest at the abandoned address that advertises the current version and points at the current

files, which is a redirection in everything but name. Testers on the beta channel now update in place.

The lesson generalised: the earlier failure and this one both arose from a change that did not declare what it superseded, leaving the system to infer intent from absence.

4.5 4 to 5 August: models the user chooses

Until this point the application used the model it shipped with, and one optional larger model, selected by a fixed table of filenames. A researcher who already had suitable weights — through Ollama, LM Studio, or a previous download — could not tell the application to use them.

Three modules were added. A *registry* records the models available to an installation and which task each may perform, persisted through the existing crash-safe write. A *router* answers which model serves a given task on this machine at this moment, with every dependency supplied by its caller, so that routing can be tested against fabricated hardware without a model runtime present. A *manifest*, generated during the build from the release configuration, records what that build bundled and which previously bundled models it replaces, so that exchanging the included model becomes one configuration edit rather than four coordinated source changes.

Imported models are referenced where they already are, never copied: the alternative would double the disk cost of a file measured in gigabytes on machines already selected for being constrained. A separate flag records whether the application created a file, and nothing deletes a file it did not create.

Acquisition from Hugging Face was added last, as the only part of the application that reaches the network. It searches on submission rather than on keystroke, states plainly that it is doing so, and constructs its download address from a repository identifier and a filename rather than accepting an address, since the local interface is reachable from the embedded browser.

Three defects found during this work shared a single form. A size was rounded before being compared against a memory budget, so any file below roughly five megabytes measured as zero and satisfied every constraint. A measured size of zero was treated by a default-value expression as *no measurement*, so a stale declared figure of ninety-nine gigabytes was preferred over the truth. And the memory budget itself, computed as free memory less a fixed reserve, floors at zero on a constrained machine — where zero already meant *unmeasured, therefore unconstrained*, so the machine least able to run anything was told it could run everything.

In each case a legitimate zero was indistinguishable from an absent value. The correction in each was the same: ask the module that owns the decision, which compares against real quantities, rather than testing a number for truthiness.

5 Decisions and Their Consequences

Decision	Reason	Consequence accepted
local inference only	privacy, cost, offline operation	far weaker models; seconds per response
quantised small models	fit consumer memory	reduced quality on structured output
frozen Python runtime	the user installs nothing	installers approach 1 GB
file-based vector store	no database to install	keyword index rebuilt per query
bundle only the smaller model	installer size	analysis quality depends on a consented download
bundle a typesetting engine	the writer must work with no LaTeX	70 MB added per installer
CPU-only inference build	one build behaves identically everywhere	GPU hardware unused
tags, not branches, produce releases	releases cannot happen by accident	a deliberate step is required
updates only when requested	an offline product must not contact a server unprompted	a user who never asks never updates
merges produce releases, tags record them	a stray tag must not publish to every user	releasing requires a reviewed merge
imported models referenced, never copied	a multi-gigabyte file must not cost twice the disk	the application depends on a path it does not own
task assignment chosen by the user	parameter count does not predict structured output	one extra step when adding a model
download addresses constructed, never accepted	the local interface is reachable from the embedded browser	only known hosts can be fetched from

6 What Was Learned

6.1 Testing source code is not testing a product

309 tests passed while the shipped installer could not start. The tests examined a source tree; users run an installer. The verification tiers added afterwards exercise the packaged application on each supported operating system, on machines that have no model and no typesetting distribution installed, so that a pass is evidence about a user's machine.

6.2 Suppressed errors are expensive

Two separate diagnoses were slowed by output sent to a null device. In both cases the fix was to print the error and stop, rather than warn and continue. A build that continues after a step has failed produces an artefact nobody intended to ship.

6.3 Fix categories, not instances

Three defects were the same defect. Enumerating every dependency once bounded the problem and ended it, where fixing them one at a time as testers reported them would not have converged.

6.4 Documentation drifts silently

Instructions told macOS users to use a mechanism the operating system had removed, and a LaTeX error message offered a command for one Linux distribution to every user on every platform. Both were written when they were true. Neither was noticed until a person followed them and could not proceed.

6.5 5 August: choosing a model by measuring it

The bundled model was to be replaced with something lighter and newer. Qwen3 0.6B was the obvious candidate: smaller than the model it would replace, a generation ahead, and its download verified. Every structural check passed.

It returned an empty object.

Gap finding constrains the model's output with a grammar that permits only JSON from the first token. A reasoning model begins by thinking, and the grammar leaves that nowhere to go, so it emits the shortest legal document instead: `{}`. The parser succeeds. Nothing downstream notices. The feature returns no gaps and reports no error.

A bake-off was written against the two flagship jobs, and it changed the decision entirely. The incumbent Qwen2.5 0.5B recovered four of six stated limitations, produced valid LaTeX, and was the fastest of the four candidates tested; nothing larger justified its additional weight. The model that had been about to ship on the strength of its specification came last.

One further correction belongs here, because it was the harness rather than the product. The LaTeX test initially wrapped its output in JSON to make it machine-checkable, and two candidates were marked down for producing corrupt markup — inside a JSON string, `\begin` is a valid escape and the parser silently converts it to a control character. Scribe does not do this; it asks for plain text. A test that differs from the production path measures the test.

6.6 5 August: a test that had never failed

Three times in one week the application shipped a blank page: the build succeeds, because the markup is valid, and the interface framework then throws at render time on a name that no longer exists, leaving an empty document and an error visible only in a browser console.

A test was added to mount every screen. It passed. The bug was then reproduced deliberately — the import removed again — and the test still passed, because it mounted the four page components and never the application shell, which is where the missing import was. It would have passed indefinitely while the product failed to start.

The same exercise was applied to the rest of the new work: each guarantee was broken on purpose to confirm something noticed. Two of five did not. The protection against deleting a file the application did not create had a clear error message, an obvious intent, and no test at all.

The general lesson is worth stating plainly, because this project has now learned it twice in different forms. A suite that has never failed is a hypothesis. Sprint II established that testing source code is not testing a product; this establishes that writing a test is not the same as being protected by one.

7 Division of Work

Work was divided by feature rather than by layer, so that each member owned a capability end to end — its data model, its interface, and its failure modes — rather than a horizontal slice of everybody's. The consequence is that no part of the product can be described without naming the person who built it.

Capability	Principal author	Scope
Retrieval and knowledge base	Aditya	document ingestion, chunking, embedding index, semantic search
Analysis	Aditya	summarisation, claim extraction, theme clustering, and the precomputation that moved them off the request path
Gap finder	Aditya	gap detection over the analysed corpus, and the suggestion engine above it
LitGraph	Aditya	the canvas consolidating search, analysis and gaps; the reader, passage marking and node interaction
Scribe	Rithesh	LaTeX editor, the auto-healing compiler, and the natural language to markup path
Desktop shell	Rithesh	the Tauri application, native hardware diagnosis, process supervision, in-application updates
Delivery engineering	Rithesh	cross-platform build, release pipeline, artefact validation, and the automated test suite
Model management	Rithesh	hardware capability model, task routing, the model registry, and Hugging Face acquisition
Encryption	Jitvan	at-rest encryption and decryption of stored documents, with the key handling and interface around it
Distribution site	Jitvan	the public landing and download page
Interface refinement	Jitvan	typography and identity, dark mode correction, and the assistant panel

The boundaries were not absolute. The interface was worked on by all three, and several features crossed hands: encryption was introduced by Jitvan and later extended when the key derivation was hardened; the desktop shell was begun as packaging work and grew into hardware diagnosis, which then fed the model routing. Where a capability changed owner, the original author's design was retained and the reasoning recorded in the decision log rather than replaced silently.

Review was mutual and enforced by the branch rules described above: no change reached the release branches without passing another member's review and the automated gate, including changes by the member who wrote those rules.

8 Present State

Measure	Value
Commits	237
Pull requests merged	43
Application code (Python)	11 686 lines, 74 modules
Test code	5 761 lines, 34 modules, 643 tests
Interface tests	63 tests, including every screen mounted
Interface code	5 781 lines, 14 components
Desktop shell (Rust)	777 lines
Build and release tooling	3 439 lines, 18 scripts, 6 workflows
REST endpoints	48
Architecture decisions recorded	40
Releases published	6, most recently beta v1.9.0
Platforms built and validated automatically	3

Figures are measured on the working branch at 5 August and therefore include the model management work of 4 to 5 August, which is complete and tested but not yet merged. The last

released build, beta v1.9.0, carries 8 725 lines of application code and 474 tests; the difference between the two figures is that work.

8.1 Verification status by platform

Platform	Status
Linux	automatically validated and exercised by hand on physical hardware
Windows	automatically validated and confirmed working by a human tester, though not yet across a range of machines
macOS	automatically validated and exercised by a tester; unsigned applications are still obstructed on first launch
In-application update	working; testers update in place from the beta channel

9 Planned Work

The following are planned rather than aspirational. Each follows from a limitation identified during development, and several are already partly prepared for in the existing structure of the system.

9.1 Language models trained for this application

The clearest quality limitation is the capability of a general-purpose 0.5 billion parameter model on structured output. Asked to plot a function it produced a reference to an image file that does not exist, where a 1.5 billion parameter model produced a correct plot. The intended response is models suited to the task rather than simply larger ones, in three stages of increasing ambition.

Stage one: fine-tuned task models. Training pairs of natural language instruction and compilable LaTeX are already collected passively during normal use, and the routing table in the inference client already contains entries for a dedicated writing model, so adopting one is a matter of supplying weights. A second model fine-tuned for the structured summarisation and claim extraction formats the application parses would address the cases where a general model returns prose where JSON was requested.

Stage two: adapter distribution. Rather than shipping whole models, distribute small adapters applied over the bundled base model, so a capability upgrade costs tens of megabytes instead of gigabytes. Selection would use the hardware profile the application already computes at startup, so a machine receives only variants it can run.

Stage three: models trained from scratch. The longer term intention is to train small language models specifically for academic literature work, rather than adapting general-purpose ones. A model trained from the outset on research text, structured extraction and typesetting markup can be far smaller than a general model of equivalent usefulness for these tasks, which matters directly for a product constrained to consumer hardware.

Federated learning. A possible direction, subject to careful design, is improving these models from real usage without collecting anyone's documents. Training would occur on the user's own machine and only model updates, never text, would leave the device, and only with explicit consent. This is recorded as a possibility rather than a commitment: it must not be allowed to weaken the privacy guarantee that is the reason the product exists, and that constraint takes precedence over the capability.

9.2 An interface design philosophy, and the refactor that follows it

The interface was built feature by feature under time pressure. It is coherent but was not designed as a system: styling has accumulated into a single large stylesheet with two competing styling mechanisms in use.

The intention is deliberately ordered. A design philosophy will first be agreed by the team through discussion, and the interface will then be refactored to it. Refactoring before that agreement would simply produce a second set of accumulated decisions. Points to be settled include:

- **Design tokens.** Colour, spacing and typography expressed as variables, so the interface compiles from a defined system rather than from decisions made one screen at a time.
- **One environment rather than four.** Browsing, searching, interpreting and writing are currently separate screens. The intended direction is a single workspace whose density and visual quiet change with the task.
- **A single cross-platform identity** rather than per-platform conventions, so the application looks deliberate on every operating system.
- **Latency as a design material.** Local inference takes seconds and that cannot be removed. The interface should be designed around honest waiting rather than concealing it.
- **Reading comfort.** Sessions are long and the content is dense, so typography, measure and contrast are functional requirements rather than decoration.

9.3 A robust paper editor

The paper writer is the most used feature and has received the most correction, but its editing surface remains a plain text area over LaTeX source. The planned work is a genuinely capable editor:

- syntax awareness, bracket matching, and compiler diagnostics shown against the line that caused them;
- a visual editing mode in which document structure can be edited without the author reading markup, with the compiled PDF remaining the single source of truth for appearance;
- position synchronisation between the source and the compiled document, so that selecting a place in one moves to it in the other;
- continued strengthening of the natural language to LaTeX path, which currently rewrites a selected passage in place and is the feature the planned writing model would most improve.

9.4 Further planned refactors

- **Incremental keyword index**, removing the per-query rebuild that currently limits corpus size.
- **Stylesheet consolidation**, resolving the two styling mechanisms into one, carried out as part of the interface refactor above rather than separately.
- **Code signing and notarisation** for macOS and Windows. This is required before a production release: both operating systems currently obstruct the first launch of an unsigned application, which every user encounters and which reads as the software being broken.
- **Bundled model integrity checks**, so that a corrupted or partial payload is detected at startup rather than at first use.

10 Closing Note

The technically demanding parts of this project, namely local inference with hardware-aware model selection, hybrid retrieval, an auto-healing typesetting pipeline and authenticated encryption, were completed and work. What determined whether anyone else could use the software was packaging, dependency completeness, and verification of the delivered artefact.

The distinction between software that works and software that has been shown to work on a machine other than its author's is the lesson this project taught most clearly, and it was taught by a release that failed rather than by one that succeeded.